

Modelado supervisado

Entrenar y evaluar modelos con criterio

BLOQUE 0 · Enfoque general de la sesión

Qué vas a hacer realmente en esta sesión

Hasta ahora, en el curso, has trabajado principalmente en **entender los datos**:

- analizarlos,
- visualizarlos,
- detectar patrones,
- prepararlos para un posible modelado.

En esta sesión se produce un cambio decisivo:

por primera vez vas a entrenar, evaluar y comparar modelos supervisados completos, de principio a fin.

No se trata de “probar un algoritmo”, sino de **cerrar el ciclo real del aprendizaje supervisado**, incorporando criterio y capacidad de decisión.

En concreto, en esta sesión vas a:

- formular correctamente un problema supervisado,
- entrenar **varios modelos distintos** sobre el mismo conjunto de datos,
- evaluar su rendimiento con **métricas de clasificación bien interpretadas**,
- analizar sus errores mediante visualización,
- y **comparar modelos con criterio**, no solo con números.

Por qué esta sesión es distinta a las anteriores

Las sesiones anteriores se centraban en preparar el terreno:

datos, variables, visualización y razonamiento previo.

La sesión 6 es distinta porque:

- introduces modelos que **toman decisiones automáticamente**,
- empiezas a enfrentarte a **errores reales del modelo**,
- y debes justificar por qué un modelo es preferible a otro.

Además, esta es la **última sesión dedicada al aprendizaje supervisado clásico**.

A partir de aquí, el curso cambia de naturaleza:

- en la **sesión 7**, desaparecerá la variable objetivo (aprendizaje no supervisado),
- en la **sesión 8**, aparecerán modelos mucho menos interpretables (redes neuronales),
- en las **sesiones 9 y 10**, el foco estará en la evaluación avanzada, la generalización y el criterio profesional.

Por eso, esta sesión no puede quedarse solo en la teoría:

aquí se consolida todo lo aprendido hasta ahora.

Qué NO vas a hacer (y por qué)

Esta sesión **no** tiene como objetivo:

- ajustar hiperparámetros de forma exhaustiva,
- buscar la mejor métrica posible,
- optimizar modelos “por probar”,
- ni convertir el trabajo en una competición de resultados.

Estas decisiones son deliberadas.

Antes de optimizar, necesitas saber:

- qué hace un modelo,
- cómo se equivoca,
- qué tipo de error comete,
- y **qué métricas son relevantes según el problema**.

Todo eso se aprende **antes** de entrar en afinados finos.

Qué modelos y herramientas vas a utilizar

A lo largo de la sesión trabajarás con **varios modelos supervisados representativos**, cada uno con un propósito pedagógico claro:

- un modelo trivial (para establecer el mínimo),
- un modelo lineal e interpretable,
- un modelo basado en distancia,
- un modelo no lineal,
- y un modelo de tipo ensamble.

No se trata de memorizar algoritmos, sino de **entender cómo cambia el comportamiento del modelo** según su naturaleza.

Cómo se van a usar las métricas en esta sesión

Las métricas de clasificación (accuracy, precision, recall, F1-score) no se introducirán como definiciones aisladas.

En esta sesión se utilizarán como:

- **herramientas de interpretación del error,**
- **criterios de comparación entre modelos,**
- **y apoyo a la toma de decisiones.**

Para ello, encontrarás a lo largo del bloque final:

- **cuadros resumen** que explican qué mide realmente cada métrica,
- **cuadros de decisión** que ayudan a interpretar resultados típicos,
- y ejemplos donde una métrica aparentemente “buena” no implica un modelo adecuado.

Qué deberías ser capaz de hacer al terminar la sesión

Al finalizar esta sesión deberías poder afirmar, con criterio, que sabes:

- entrenar varios modelos supervisados distintos,
- evaluarlos correctamente con métricas adecuadas,
- interpretar una matriz de confusión,
- comparar modelos bajo las mismas condiciones,
- y justificar una elección **sin basarte solo en un número.**

Ese es el nivel mínimo necesario para avanzar hacia modelos más complejos sin convertirlos en cajas negras.

Cómo se estructura la sesión a partir de aquí

La sesión se organiza en cinco bloques, cada uno con un propósito claro:

- **BLOQUE 1** · Del dato preparado al problema entrenable
- **BLOQUE 2** · Definición formal del problema supervisado
- **BLOQUE 3** · Generalización: entrenar no es evaluar
- **BLOQUE 4** · Modelos supervisados: baseline y alternativas reales
- **BLOQUE 5** · Evaluar y comparar modelos: métricas, visualización y decisión

Cada bloque construye una parte del razonamiento completo.

No están pensados para leerse de forma independiente.

Están pensados para leerse en secuencia, porque cada bloque reutiliza decisiones del anterior.

Idea clave para empezar la sesión

Esta sesión no va de aprender “muchos modelos”.

Va de aprender **a decidir correctamente cuando entrenas modelos supervisados**.

Con esta idea clara, entramos ya en el contenido técnico.

BLOQUE 1 · Del dato preparado al problema entrenable

1.1 Cambio de rol: de analizar datos a construir modelos

Hasta este punto del curso, tu trabajo principal ha sido **comprender los datos**:

- explorar distribuciones,
- detectar patrones,
- identificar valores atípicos,
- formular hipótesis razonables.

Ese trabajo no desaparece en esta sesión, pero **cambia su función**.

A partir de ahora, los datos dejan de ser solo un objeto de análisis y se convierten en **materia prima para un modelo que toma decisiones automáticamente**.

Este cambio implica un nuevo rol:

ya no basta con entender los datos;
ahora debes ser capaz de **hacer que un modelo actúe sobre ellos** y justificar ese comportamiento.

1.2 Qué significa realmente “entrenar un modelo”

En el uso cotidiano de librerías de machine learning, entrenar un modelo suele reducirse a una instrucción como:

```
modelo.fit(X, y)
```

Sin embargo, desde el punto de vista conceptual, entrenar un modelo supervisado implica mucho más:

- existe un **problema bien definido**,
- existe una **variable objetivo explícita**,
- existe una **relación desconocida** entre entradas y salida,
- y el modelo intenta **aproximar esa relación** a partir de ejemplos conocidos.

Entrenar un modelo significa, por tanto:

ajustar internamente una estructura matemática
para que produzca salidas coherentes con ejemplos etiquetados,
con la expectativa de que ese comportamiento se mantenga en datos nuevos.

Este ajuste no es neutro ni automático:

está condicionado por el tipo de modelo, la calidad de los datos y las decisiones previas.

1.3 De datos preparados a datos entrenables

No todos los datos “bien preparados” son automáticamente **entrenables**.

Para que un conjunto de datos pueda alimentar un modelo supervisado, deben cumplirse condiciones muy concretas:

- las variables deben estar en un formato numérico utilizable,
- debe existir una **variable objetivo clara y separada**,

- las observaciones deben ser coherentes entre sí,
- no debe haber información que “revele” directamente la respuesta.

En otras palabras:

preparar datos para analizarlos

no es exactamente lo mismo que prepararlos para entrenar un modelo.

Este matiz es crucial, porque muchos problemas de modelado no provienen del algoritmo, sino de **suposiciones incorrectas sobre los datos de entrada**.

1.4 El puente entre análisis y modelado

Este bloque actúa como un **puente conceptual** entre dos formas de trabajar con datos:

Análisis de datos	Modelado supervisado
Visualizar para entender	Entrenar para predecir
Formular hipótesis	Ajustar un modelo
Detectar patrones	Generalizar relaciones
Interpretar gráficos	Evaluar errores

A partir de ahora, cualquier decisión tomada en el análisis previo tendrá una **consecuencia directa** en el comportamiento del modelo.

Un error conceptual que antes solo afectaba a la interpretación, ahora afecta a **decisiones automáticas**.

1.5 Preguntas de control antes de entrenar

Antes de entrenar cualquier modelo supervisado, deberías ser capaz de responder de forma explícita a estas preguntas:

1. **¿Qué estoy intentando predecir exactamente?**

(una clase, un valor continuo, una probabilidad)

2. **¿Con qué información lo intento predecir?**

(qué variables entran realmente en el modelo)

3. **¿Qué significa “hacerlo bien” en este problema?**

(qué tipo de error es más grave)

4. **¿Cómo sabré si el modelo ha aprendido o solo ha memorizado?**

Estas preguntas **no pertenecen al algoritmo**, sino al razonamiento previo.

Un modelo entrenado sin responderlas puede funcionar técnicamente y, aun así, ser inútil o engañoso.

1.6 Anticipando lo que viene en los siguientes bloques

Este bloque no introduce todavía modelos concretos ni métricas, pero prepara el terreno para todo lo que sigue:

- en el **BLOQUE 2**, se formalizará el problema supervisado (X , y , tipos de objetivo),
- en el **BLOQUE 3**, se abordará la generalización y la necesidad de evaluar correctamente,
- en el **BLOQUE 4**, se entrenarán varios modelos supervisados reales,
- y en el **BLOQUE 5**, se evaluarán y compararán esos modelos con métricas y visualización.

Cada paso se apoya directamente en las decisiones que se toman aquí.

1.7 Idea clave del bloque

Un modelo supervisado no empieza cuando se llama a `.fit()`.

Empieza cuando decides **qué problema estás resolviendo**
y **qué significa resolverlo bien**.

Sin esa claridad, ningún modelo —por sofisticado que sea— puede aportar valor real.

Referencias específicas · BLOQUE 1

- Documentación oficial de **scikit-learn** — *Supervised learning overview*
https://scikit-learn.org/stable/supervised_learning.html
 - Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*
Capítulos 1 y 2
 - Kuhn, M., Johnson, K. *Applied Predictive Modeling*
Capítulo 1 (Foundations)
-

BLOQUE 2 · Definición formal del problema supervisado

2.1 El problema antes que el algoritmo

Uno de los errores más frecuentes al iniciarse en aprendizaje automático es **empezar el proceso eligiendo un algoritmo**:

- “probemos una regresión logística”
- “probemos un árbol”
- “probemos una red”

Este enfoque invierte el orden correcto de razonamiento.

En un planteamiento riguroso, el orden siempre es el siguiente:

1. Definir el **problema real**
2. Traducirlo a un **problema supervisado concreto**
3. Identificar claramente entradas y salida
4. Decidir cómo se va a evaluar el resultado
5. Solo entonces, elegir modelos adecuados

Un mismo conjunto de datos puede dar lugar a **problemas supervisados completamente distintos** dependiendo de qué se quiera predecir y con qué intención.

El algoritmo **no define el problema**.

Es el problema el que impone qué algoritmos tienen sentido.

2.2 Variables de entrada (X): qué información usa el modelo

Las **variables de entrada**, habitualmente representadas como , son el conjunto de características que el modelo utiliza para realizar sus predicciones.

Desde un punto de vista estructural:

- cada fila de  representa una observación,
- cada columna representa una característica,
- el modelo solo recibe valores numéricos o codificados.

Desde el punto de vista del modelo, es importante entender que:

- no existe contexto,

- no existe significado semántico,
- no existe jerarquía entre variables.

El modelo **no sabe** si una variable es:

- una edad,
- un precio,
- una categoría codificada,
- o un identificador mal introducido.

Por eso, la definición de **x** es una **decisión de diseño**, no una consecuencia automática del dataset.

2.3 Variable objetivo (y): qué intenta aprender el modelo

La **variable objetivo**, representada como **y**, es aquello que el modelo intenta predecir.

Debe cumplir condiciones muy estrictas:

- estar claramente definida,
- ser conocida durante el entrenamiento,
- representar exactamente el fenómeno que se quiere modelar,
- no aparecer directa ni indirectamente dentro de **x**.

Según el problema, **y** puede ser:

- una etiqueta de clase,
- un valor continuo,
- una categoría binaria o multiclase.

Un error en la definición de **y** **no se corrige con mejores modelos**.

Si el objetivo está mal planteado, todo el proceso posterior queda invalidado.

2.4 Pares de aprendizaje: la relación $X \rightarrow y$

El aprendizaje supervisado se basa en **pares de entrenamiento**:

| a cada conjunto de entradas (X) le corresponde una salida conocida (y).

El modelo observa muchos ejemplos de este tipo y ajusta internamente su comportamiento para que, ante entradas similares, produzca salidas coherentes.

Este esquema es común a:

- modelos lineales,
- modelos basados en distancia,
- árboles de decisión,
- redes neuronales.

Lo que cambia entre modelos **no es el esquema**, sino **cómo representan internamente esa relación**.

Entender esto es clave para no pensar que cada modelo “aprende cosas distintas”: todos intentan aproximar la misma relación, con herramientas diferentes.

2.5 Tipos fundamentales de problemas supervisados

2.5.1 Clasificación

En un problema de **clasificación**, la variable objetivo toma valores **discretos**.

Características clave:

- el objetivo pertenece a un conjunto finito de clases,
- el modelo aprende fronteras de decisión,
- el error no se mide como distancia numérica simple.

Ejemplos habituales:

- spam / no spam
- fraude / no fraude
- aprobado / suspenso
- tipo de cliente

Este tipo de problema condiciona:

- los modelos que tienen sentido,
 - las métricas que se utilizarán,
 - la interpretación de los errores.
-

2.5.2 Regresión

En un problema de **regresión**, la variable objetivo es un **valor continuo**.

Características clave:

- la salida es numérica,
- el modelo aprende una función aproximada,
- el error se mide como desviación entre valores.

Ejemplos habituales:

- precio de una vivienda,
- demanda estimada,
- tiempo de entrega.

Aunque el flujo general es similar al de clasificación, **las métricas y los modelos cambian**, por lo que mezclar ambos tipos de problema conduce a errores graves.

2.6 Errores conceptuales frecuentes al definir X e y

En esta fase aparecen errores especialmente peligrosos porque **no generan errores de código**, pero sí modelos engañosos:

- incluir la variable objetivo dentro de `X`,
- utilizar variables que contienen información del futuro,
- definir un objetivo ambiguo o mal alineado con el problema real,
- cambiar el objetivo sin replantear todo el pipeline,
- usar identificadores como si fueran variables predictivas.

Estos errores suelen manifestarse más tarde como:

- métricas sorprendentemente buenas,
- modelos que “funcionan demasiado bien”,
- o resultados que no se sostienen fuera del dataset.

Detectarlos aquí ahorra muchos problemas después.

2.7 Relación con las herramientas de machine learning

Librerías como **scikit-learn** formalizan este planteamiento de forma explícita:

- `X` como matriz de características,
- `y` como vector objetivo,

- separación clara entre entrenamiento y predicción.

Esta estructura no es un detalle técnico, sino la **traducción directa del razonamiento conceptual** que se está construyendo en este bloque.

2.8 Idea clave del bloque

En aprendizaje supervisado, la mayor parte de los errores importantes no están en el algoritmo, sino en **cómo se ha definido el problema**.

Definir bien x , y y el tipo de problema es lo que permite que, más adelante, las métricas y las comparaciones tengan sentido.

Referencias específicas · BLOQUE 2

- Documentación oficial de scikit-learn — *Supervised learning*
https://scikit-learn.org/stable/supervised_learning.html
 - Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*
Capítulo 2
 - Kuhn, M., Johnson, K. *Applied Predictive Modeling*
Capítulo 2 (Data and Target Definition)
-

BLOQUE 3 · Generalización: entrenar no es evaluar

3.1 Por qué entrenar y evaluar no son la misma cosa

Cuando se empieza a trabajar con modelos supervisados, es muy habitual confundir dos acciones distintas:

- **entrenar un modelo,**
- **evaluar un modelo.**

Entrenar consiste en ajustar un modelo utilizando ejemplos cuyos resultados son conocidos.

Evaluar consiste en comprobar **cómo se comporta ese modelo ante datos nuevos**.

Si ambas acciones se realizan sobre los mismos datos, el resultado no mide la capacidad predictiva real del modelo, sino su capacidad de **recordar**.

Desde un punto de vista conceptual:

- un modelo no se evalúa para saber si ha aprendido los datos,
- sino para saber si puede enfrentarse a datos que nunca ha visto.

Esta distinción es uno de los pilares del aprendizaje automático moderno.

3.2 Generalización como objetivo real del aprendizaje supervisado

El objetivo último de un modelo supervisado no es ajustarse perfectamente a los datos de entrenamiento, sino **generalizar**.

Un modelo generaliza bien cuando:

- aprende patrones reales presentes en los datos,
- ignora el ruido específico del conjunto de entrenamiento,
- mantiene un comportamiento razonable en datos nuevos.

Por el contrario, un modelo falla cuando:

- memoriza ejemplos concretos,
- se adapta a irregularidades accidentales,
- pierde capacidad predictiva fuera del conjunto con el que fue entrenado.

Este equilibrio entre aprender y no memorizar es el núcleo del aprendizaje supervisado y condiciona todas las decisiones posteriores.

3.3 Sobreajuste: cuando aprender demasiado es un problema

El **sobreajuste** (overfitting) aparece cuando un modelo se adapta en exceso a los datos de entrenamiento.

Conceptualmente, ocurre cuando el modelo:

- aprende no solo los patrones relevantes,
- sino también el ruido y las particularidades accidentales del dataset.

Esto suele manifestarse como:

- rendimiento muy alto sobre los datos de entrenamiento,
- rendimiento significativamente peor sobre datos nuevos.

Las causas más habituales del sobreajuste incluyen:

- modelos demasiado complejos para la cantidad de datos disponible,
- conjuntos de datos pequeños,
- variables con mucho ruido,
- ausencia de una evaluación adecuada.

Es importante entender que:

un modelo sobreajustado no es un buen modelo,
aunque "explique" perfectamente los datos con los que fue entrenado.

3.4 Separación de datos: entrenamiento y evaluación

Para evaluar correctamente un modelo supervisado, los datos se dividen en conjuntos con funciones distintas.

3.4.1 Conjunto de entrenamiento

El conjunto de entrenamiento:

- se utiliza para ajustar los parámetros del modelo,
- influye directamente en lo que el modelo aprende,
- representa los ejemplos conocidos.

El modelo tiene acceso completo a estos datos durante el proceso de entrenamiento.

3.4.2 Conjunto de evaluación (test)

El conjunto de evaluación:

- se mantiene separado durante el entrenamiento,
- no influye en los parámetros del modelo,
- se utiliza únicamente para medir rendimiento.

Este conjunto simula datos reales del futuro y permite obtener una estimación honesta de la capacidad de generalización.

3.5 Qué se protege realmente al separar los datos

Separar los datos no es un formalismo técnico, sino una **medida de protección conceptual**.

La separación protege contra:

- conclusiones engañosas,
- optimismo injustificado en métricas,
- comparaciones inválidas entre modelos,
- decisiones basadas en resultados irreales.

A partir de este punto del curso, cualquier métrica solo tiene sentido si se calcula sobre datos que **no han sido utilizados para entrenar el modelo**.

3.6 Introducción al train/test split con criterio

En la práctica, la separación de datos se realiza mediante herramientas que automatizan el proceso, como `train_test_split`, disponible en **scikit-learn**.

Su función es:

- dividir `x` e `y` de forma coherente,
- mantener alineadas entradas y salidas,
- facilitar una evaluación correcta.

Es fundamental entender que esta herramienta:

- no garantiza un buen modelo,
- no evita el sobreajuste por sí sola,
- solo implementa una decisión metodológica correcta.

El criterio sigue dependiendo del analista.

3.7 Proporciones de separación y contexto

No existe una proporción universal válida para todos los problemas.

La elección depende de factores como:

- tamaño del conjunto de datos,
- complejidad del modelo,
- necesidad de evaluación fiable.

Lo importante no es el porcentaje exacto, sino **la existencia de una evaluación honesta y separada**, coherente con el problema planteado.

3.8 Idea clave del bloque

Un modelo no se evalúa para saber si ha aprendido los datos, sino para saber si puede generalizar a datos nuevos.

Este principio será esencial en los siguientes bloques, cuando:

- se entrenen varios modelos distintos,
 - se comparen sus resultados,
 - y se empiece a hablar de límites, estabilidad y criterio profesional.
-

3.9 Visualizar el sobreajuste: cuando un modelo aprende “demasiado”

Hasta ahora, el sobreajuste se ha explicado como una idea conceptual:

un modelo se adapta muy bien a los datos con los que trabaja, pero falla cuando se enfrenta a datos nuevos.

En este punto conviene **ver ese fenómeno de forma controlada**, antes de entrar en el entrenamiento formal de modelos.

3.9.1 Un experimento visual intencionadamente simple

Para ilustrar el sobreajuste se utilizará un conjunto de datos **sintético y bidimensional**, diseñado únicamente para poder **visualizar decisiones del modelo**.

El objetivo **no es entrenar modelos reales**, ni compararlos con métricas, sino **observar cómo la complejidad afecta al comportamiento**.

Se mostrarán dos comportamientos distintos de un mismo tipo de modelo:

- un **árbol de decisión con complejidad limitada**,
- un **árbol de decisión con capacidad excesiva**.

Ambos se ajustan sobre los mismos datos, pero **no se evalúan todavía como modelos finales**.

3.9.2 Qué se observa al variar la complejidad

Al representar gráficamente las fronteras de decisión, suele aparecer un patrón muy claro:

- el modelo de baja complejidad:
 - genera fronteras simples,
 - no acierta todos los puntos,
 - pero mantiene un comportamiento estable.
- el modelo de alta complejidad:
 - genera fronteras muy irregulares,
 - se adapta casi punto a punto,
 - "recuerda" incluso el ruido del conjunto.

Esto permite observar algo clave:

aumentar la complejidad no significa aprender mejor el problema,
sino **acercarse peligrosamente a memorizar los datos**.

En el notebook se representará explícitamente el comportamiento del modelo sobre el conjunto de entrenamiento y sobre un conjunto de test separado, para visualizar la pérdida de generalización.

3.9.3 Relación directa con la generalización

Este experimento visual conecta directamente con lo visto en este bloque:

- entrenar y evaluar no son la misma cosa,
- un comportamiento perfecto sobre los datos conocidos es sospechoso,
- la generalización es el objetivo real del aprendizaje supervisado.

El sobreajuste **no es un error de implementación**,

sino una consecuencia natural de permitir demasiada flexibilidad al modelo.

Cuadro de lectura rápida · Reconocer sobreajuste en la práctica

Señal observable	Interpretación	Acción típica
Ajuste perfecto a los puntos	Memorización	Reducir complejidad
Fronteras muy irregulares	Sigue ruido	Regularizar
Cambios mínimos alteran mucho	Alta varianza	Ensamblar / más datos
Comportamiento "demasiado bueno"	Sospechoso	Revisar planteamiento

3.9.4 Por qué esto aparece antes del BLOQUE 4

Este apartado **no introduce todavía entrenamiento formal**, pero prepara el criterio necesario para entender:

- por qué algunos modelos funcionan peor en test,
- por qué la complejidad tiene un coste,
- y por qué, en el siguiente bloque, se hablará de **estabilidad y ensambles**.

Idea clave del subapartado

Antes de aprender a entrenar modelos, necesitas aprender a **desconfiar de los modelos que parecen perfectos**.

Referencias específicas · BLOQUE 3

- Documentación oficial de scikit-learn — *Training and test data*
https://scikit-learn.org/stable/model_selection.html#training-and-test-data
 - Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*
Capítulo 2 (Train/Test Split y Generalization)
 - Kuhn, M., Johnson, K. *Applied Predictive Modeling*
Capítulo 4 (Overfitting and Model Evaluation)
-

BLOQUE 4 · Modelos supervisados: baseline y alternativas reales

4.1 Por qué no empezar directamente por “un buen modelo”

Una vez definido el problema (BLOQUE 2) y establecida la necesidad de evaluar correctamente (BLOQUE 3), aparece una tentación habitual:

| entrenar directamente un modelo “potente”.

Este enfoque es un error metodológico.

Antes de hablar de modelos “mejores”, es imprescindible establecer **referencias**.

Por eso, el trabajo con modelos supervisados **siempre empieza con un baseline**.

Un baseline permite responder a una pregunta fundamental:

¿Este problema se puede predecir mejor que una solución trivial?

Sin baseline:

- no hay criterio de mejora,
- no hay comparación honesta,
- no se puede justificar la complejidad posterior.

4.2 Baseline trivial: establecer el suelo del problema

4.2.1 Qué es un baseline trivial

Un **baseline trivial** es un modelo que no aprende relaciones reales, sino que aplica una regla extremadamente simple.

Ejemplos de estrategias triviales:

- predecir siempre la clase más frecuente,
- predecir al azar siguiendo la distribución de clases.

Su función **no es acertar**, sino marcar el **mínimo rendimiento aceptable**.

4.2.2 Entrenamiento de un baseline trivial

Ejemplo con un clasificador que siempre predice la clase más frecuente:

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier

# Datos
X, y = load_breast_cancer(return_X_y=True)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Baseline trivial
dummy = DummyClassifier(strategy="most_frequent")
```

```
dummy.fit(X_train, y_train)

y_pred_dummy = dummy.predict(X_test)
```

Este modelo sirve como **línea base**:

cualquier modelo “serio” debería superarlo claramente.

4.3 Baseline razonable: Regresión Logística

4.3.1 Por qué la regresión logística es un buen punto de partida

Aunque su nombre pueda confundir, la **regresión logística** es un **modelo de clasificación**.

Desde el punto de vista pedagógico es ideal porque:

- es lineal,
- es interpretable,
- introduce la idea de probabilidad,
- y suele generalizar razonablemente bien.

No es un modelo “antiguo”: sigue siendo ampliamente usado en producción.

4.3.2 Entrenamiento de una regresión logística

```
from sklearn.linear_model import LogisticRegression

log_reg = LogisticRegression(max_iter=5000)
log_reg.fit(X_train, y_train)

y_pred_log = log_reg.predict(X_test)
```

Este modelo suele mejorar claramente al baseline trivial y se convierte en la **referencia principal** frente a la cual comparar otros enfoques.

4.4 Modelo basado en distancia: k-Nearest Neighbors (k-NN)

4.4.1 Qué cambia respecto a un modelo lineal

El algoritmo **k-NN** clasifica una observación en función de las clases de sus vecinos más cercanos.

Esto introduce una idea clave:

la predicción depende de la geometría del espacio de datos.

No aprende una función explícita, sino que:

- compara distancias,
- decide por proximidad.

4.4.2 Advertencia conceptual importante

k-NN es muy sensible a:

- escalas distintas entre variables,
- variables irrelevantes,
- ruido en los datos.

Esto lo convierte en un excelente modelo **didáctico** para entender por qué las decisiones previas sobre los datos importan tanto.

4.4.3 Entrenamiento de k-NN

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)
```

El comportamiento de este modelo suele diferir claramente del modelo lineal, incluso usando los mismos datos.

4.5 Modelo no lineal interpretable: Árbol de decisión

4.5.1 Qué aporta un árbol de decisión

Un **árbol de decisión** aprende reglas del tipo:

“si esta variable cumple esta condición, ve por aquí; si no, por allá”.

Aporta:

- relaciones no lineales,
- interpretabilidad estructural,
- decisiones explícitas.

4.5.2 Riesgo principal: sobreajuste fácil

Los árboles tienen una característica peligrosa:

| pueden ajustarse “demasiado bien” a los datos de entrenamiento.

Por eso, un árbol con resultados espectaculares debe analizarse **con cautela**, especialmente en conjuntos de datos pequeños.

4.5.3 Entrenamiento de un árbol de decisión

```
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)

y_pred_tree = tree.predict(X_test)
```

Este modelo suele mostrar un comportamiento distinto a los anteriores, especialmente en términos de error.

4.6 Ensamble para estabilidad: Random Forest

4.6.1 Qué idea introduce un ensamble

Un **Random Forest** combina muchos árboles de decisión independientes y agrega sus predicciones.

La idea clave es:

| muchos modelos simples e inestables
| pueden producir un resultado más estable cuando se combinan.

Esto reduce:

- la varianza,

- el riesgo de sobreajuste extremo.

4.6.2 Entrenamiento de un Random Forest (sin tuning)

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
```

Aquí **no se optimiza nada**:

el objetivo es comparar comportamientos, no exprimir rendimiento.

4.7 Cuadro resumen · Modelos vistos y qué aportan

Modelo	Tipo	Qué aporta	Riesgo principal
Dummy	Trivial	Marca el suelo	Ninguno (no aprende)
Regresión logística	Lineal	Interpretabilidad	No capta no linealidad
k-NN	Distancia	Sensible a geometría	Escalas, ruido
Árbol de decisión	No lineal	Reglas claras	Sobreajuste
Random Forest	Ensamble	Estabilidad	Menor interpretabilidad

Este cuadro servirá como referencia directa en el **BLOQUE 5**, cuando se comparen métricas y errores.

4.8 Idea clave del bloque

Los modelos no “compiten” entre sí.

Cada uno representa una **forma distinta de entender los datos**.

Compararlos no consiste en elegir el más alto en una métrica, sino en **entender qué tipo de errores comete cada uno**.

Referencias específicas · BLOQUE 4

- Documentación oficial de scikit-learn
 - DummyClassifier
<https://scikit-learn.org/stable/modules/generated/sklearn.dummy.DummyClassifier.html>
 - LogisticRegression
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
 - KNeighborsClassifier
<https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>
 - DecisionTreeClassifier
<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
 - RandomForestClassifier
<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
-

BLOQUE 5 · Evaluar y comparar modelos: métricas, visualización y decisión

5.1 Evaluar no es “mirar un número”

Una vez entrenados varios modelos (BLOQUE 4), el siguiente paso **no** es elegir el que tenga la métrica más alta sin más.

Evaluar un modelo significa responder preguntas como:

- ¿Qué tipo de errores comete?
- ¿Dónde falla más?
- ¿Esos errores son aceptables para el problema?
- ¿Mejora realmente a las alternativas?

Las métricas **resumen** el comportamiento del modelo, pero **no lo explican por sí solas**.

Para evaluar con criterio es necesario entender **qué errores penaliza cada métrica** y **qué información deja fuera**.

5.2 Métricas de clasificación: qué mide realmente cada una

Antes de calcular nada, es fundamental entender **qué pregunta responde cada métrica**, no cómo se calcula.

Cuadro resumen · Métricas de clasificación

Métrica	Qué responde	Cuándo es clave	Riesgo si se usa sola
Accuracy	¿Cuántas predicciones totales son correctas?	Clases equilibradas	Oculto clases minoritarias
Precision	¿De los positivos predichos, cuántos eran correctos?	Falsos positivos costosos	Ignora positivos reales
Recall	¿De los positivos reales, cuántos detecta?	Falsos negativos críticos	Genera muchos falsos positivos
F1-score	¿Hay equilibrio entre precision y recall?	Clases desbalanceadas	Oculto qué error importa más

Idea clave: no existe una "mejor métrica" universal.

La métrica correcta depende del problema y de las consecuencias del error.

5.2.1 Caso de decisión: cuando la accuracy no es suficiente

Una vez entendidas las métricas de clasificación, es momento de enfrentarse a una decisión real.

No se trata de calcular números, sino de **elegir un modelo con consecuencias**.

5.2.1.1 El contexto del problema

Imagina un sistema de detección de una **enfermedad rara**:

- solo el **5%** de los pacientes la padecen,
- el sistema se usa como **cribado inicial**,
- fallar en detectar un caso real tiene consecuencias graves.

Se entrenan dos modelos distintos sobre el mismo conjunto de datos.

5.2.1.2 Resultados de ambos modelos

Sobre 1000 pacientes:

Modelo A

- Accuracy: **95%**
- Detecta pocos casos reales

Matriz de confusión (simplificada y diseñada para forzar el dilema de decisión; no pretende ser un ejemplo estadístico perfecto)::

- TP = 10
- FN = 40
- FP = 10
- TN = 940

Modelo B

- Accuracy: **90%**
- Detecta casi todos los casos reales

Matriz de confusión (simplificada):

- TP = 45
- FN = 5
- FP = 120
- TN = 830

5.2.1.3 Preguntas que obligan a decidir

Antes de mirar ninguna métrica "resumen", plantéate:

1. ¿Qué modelo dejaría **menos pacientes enfermos sin detectar**?
2. ¿Qué error es más grave en este contexto: un falso positivo o un falso negativo?
3. ¿Qué modelo preferirías usar como **cribado inicial**?
4. ¿Cambiaría tu elección si el test posterior fuese muy caro o invasivo?

No hay una respuesta "correcta" universal.

Hay una respuesta **justificada por el contexto**.

5.2.1.4 Traducción del dilema a métricas

Este caso permite entender algo fundamental:

- el **Modelo A** optimiza accuracy,
- el **Modelo B** optimiza recall.

La elección depende de **qué error estás dispuesto a aceptar**.

Cuadro resumen · Contexto → métrica prioritaria

Contexto	Error más grave	Métrica prioritaria
Cribado médico	Falso negativo	Recall
Prueba confirmatoria cara	Falso positivo	Precision
Clases equilibradas	Ambos similares	Accuracy
Clases desbalanceadas	Accuracy engañosa	F1 + matriz

5.2.1.5 Cierre conceptual del caso

Este ejemplo ilustra el principio central del bloque:

Un modelo no es mejor por acertar más,
sino por **equivocarse de la forma correcta** para el problema que resuelve.

Ese es el criterio profesional que se busca desarrollar en esta sesión.

5.3 De la matriz de confusión a las métricas (formulación explícita)

Todas las métricas de clasificación se derivan de la **matriz de confusión**.

La matriz de confusión es la “contabilidad” del error; las métricas son distintas formas de resumir esa contabilidad.

En un problema binario, esta matriz se basa en cuatro cantidades fundamentales:

- **TP (True Positives)**: positivos correctamente predichos
- **TN (True Negatives)**: negativos correctamente predichos
- **FP (False Positives)**: negativos predichos como positivos
- **FN (False Negatives)**: positivos predichos como negativos

Estas cuatro cantidades describen **todos los errores posibles** del modelo.

Accuracy (exactitud global)

La **accuracy** mide la proporción total de predicciones correctas:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Qué penaliza: todos los errores por igual.

Qué ignora: el reparto entre clases.

Por eso, una accuracy alta **no garantiza** un buen modelo si las clases están desbalanceadas.

Precision (precisión)

La **precision** mide la fiabilidad de las predicciones positivas:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Qué penaliza: falsos positivos.

Qué ignora: falsos negativos.

Es crítica cuando **marcar algo incorrectamente como positivo** tiene un coste alto.

Recall (sensibilidad)

El **recall** mide la capacidad del modelo para detectar positivos reales:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Qué penaliza: falsos negativos.

Qué ignora: falsos positivos.

Es clave cuando **dejar pasar un positivo** es especialmente grave.

F1-score

El **F1-score** es la media armónica entre precision y recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Qué penaliza: desequilibrios extremos entre precision y recall.

Qué no decide: qué tipo de error es más importante.

Cuadro resumen · Qué penaliza realmente cada métrica

Métrica	Penaliza principalmente	Ignora parcialmente
Accuracy	FP y FN por igual	Distribución de clases
Precision	Falsos positivos	Falsos negativos
Recall	Falsos negativos	Falsos positivos
F1-score	Desequilibrios extremos	Prioridad del error

Este cuadro explica por qué **dos modelos con la misma accuracy pueden comportarse de forma muy distinta**.

5.4 Calcular métricas en la práctica

Una vez hechas las predicciones, las métricas se calculan siempre **sobre el conjunto de test**, nunca sobre entrenamiento.

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

def evaluar(y_true, y_pred):
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(y_true, y_pred),
        "recall": recall_score(y_true, y_pred),
        "f1": f1_score(y_true, y_pred),
    }

evaluar(y_test, y_pred_log)
```

En clasificación binaria, `precision_score` y `recall_score` dependen de cuál sea la clase considerada "positiva" (`pos_label`).

En esta sesión consideraremos 'positivo' el evento crítico del problema (p. ej., 'enfermedad presente'), y fijaremos explícitamente `pos_label` cuando proceda.

Para una visión más completa:

```
print(classification_report(y_test, y_pred_log))
```

Este informe permite comprobar, por ejemplo, cómo una buena accuracy puede convivir con un recall bajo.

Nota sobre el `classification_report` :

- **macro avg:** promedia las métricas por clase tratándolas por igual.
- **weighted avg:** promedia ponderando por el número de ejemplos de cada clase (soporte).

5.5 La matriz de confusión: ver los errores importa más que contarlos

Las métricas **nacen** de la matriz de confusión.

Si no entiendes esta matriz, las métricas son solo números.

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt

ConfusionMatrixDisplay.from_predictions(y_test, y_pred_log)
plt.title("Matriz de confusión · Regresión logística")
plt.show()
```

La matriz de confusión permite responder preguntas clave:

- ¿Qué clases se confunden entre sí?
- ¿Dónde se concentran los errores?
- ¿El modelo falla siempre del mismo modo?

5.6 Comparar modelos bajo las mismas condiciones

Una comparación solo es válida si **todo lo demás se mantiene constante**:

- mismo dataset,

- mismo train/test split,
- mismas métricas.

```
resultados = {
    "Dummy": evaluar(y_test, y_pred_dummy),
    "Logistic": evaluar(y_test, y_pred_log),
    "k-NN": evaluar(y_test, y_pred_knn),
    "Árbol": evaluar(y_test, y_pred_tree),
    "RandomForest": evaluar(y_test, y_pred_rf),
}

resultados
```

Aquí **no se elige automáticamente el mejor F1**.

Se analiza **cómo cambia el patrón de error** entre modelos.

5.7 Cuadro de interpretación · Leer métricas con criterio

Cuadro de decisión rápida

Observación	Interpretación probable	Qué revisar
Accuracy alta, recall bajo	El modelo "se queda corto"	Tipo de modelo, umbral
Precision alta, recall bajo	Modelo conservador	¿Es aceptable perder positivos?
Árbol mucho mejor que lineal	Relación no lineal	Riesgo de sobreajuste
Random Forest mejora a árbol	Reducción de varianza	Pérdida de interpretabilidad
Dummy cercano a modelos reales	Problema mal planteado	Variables u objetivo

Este cuadro **no da respuestas automáticas**, pero sí orienta el razonamiento.

5.8 Elegir un modelo no es optimizar, es justificar

Elegir un modelo significa poder explicar:

- por qué ese modelo y no otro,
- qué tipo de errores comete,
- qué compromisos estás aceptando,
- y por qué eso tiene sentido para el problema.

Un modelo con una métrica ligeramente peor puede ser **mejor elección** si:

- es más estable,
 - es más interpretable,
 - comete errores menos graves.
-

5.9 Idea clave del bloque (y de la sesión)

Evaluar modelos no consiste en encontrar el número más alto,
sino en **entender el comportamiento del modelo y decidir con criterio**.

Este principio es el que te permitirá:

- enfrentarte a modelos no supervisados,
 - trabajar con redes neuronales,
 - y comprender los límites reales del aprendizaje automático.
-

Referencias específicas · BLOQUE 5

- Documentación oficial de **scikit-learn** — *Classification metrics*
https://scikit-learn.org/stable/modules/model_evaluation.html#classification-metrics
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*
Capítulo 3
- Kuhn, M., Johnson, K. *Applied Predictive Modeling*
Capítulo 11